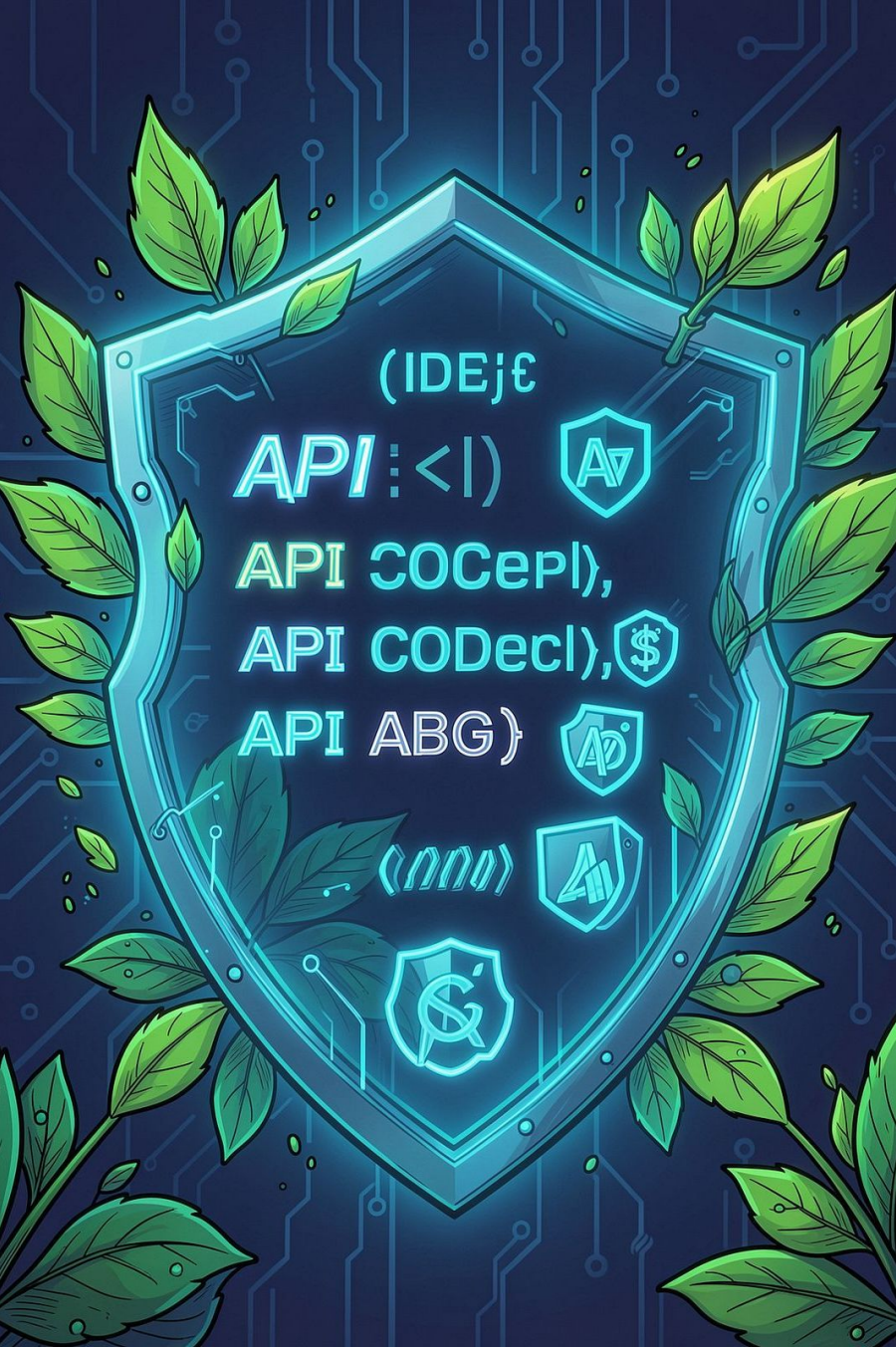




Validaciones y Manejo Global de Errores en Spring Boot

De una API Frágil a una API Profesional

SPRINT 5

CLÍNICA VETERINARIA PATITAS FELICES

El Problema: Datos Inválidos = Caos

Endpoint	Entrada inválida	Respuesta actual
POST /api/duenos	Sin campo email	HTTP 500 Internal Server Error
POST /api/turnos	Fecha en el pasado	HTTP 201 Created (¡Se guarda igual!)
POST /api/veterinarios	Matrícula vacía	HTTP 500 Internal Server Error

 Un error del cliente (HTTP 400) no debería generar un error del servidor (HTTP 500). El sistema no está protegido.

La Solución: Dos Pilares Fundamentales

Cualquier dato inválido será rechazado con un **HTTP 400 Bad Request** y un mensaje de error claro y descriptivo.



Pilar 1: Bean Validation

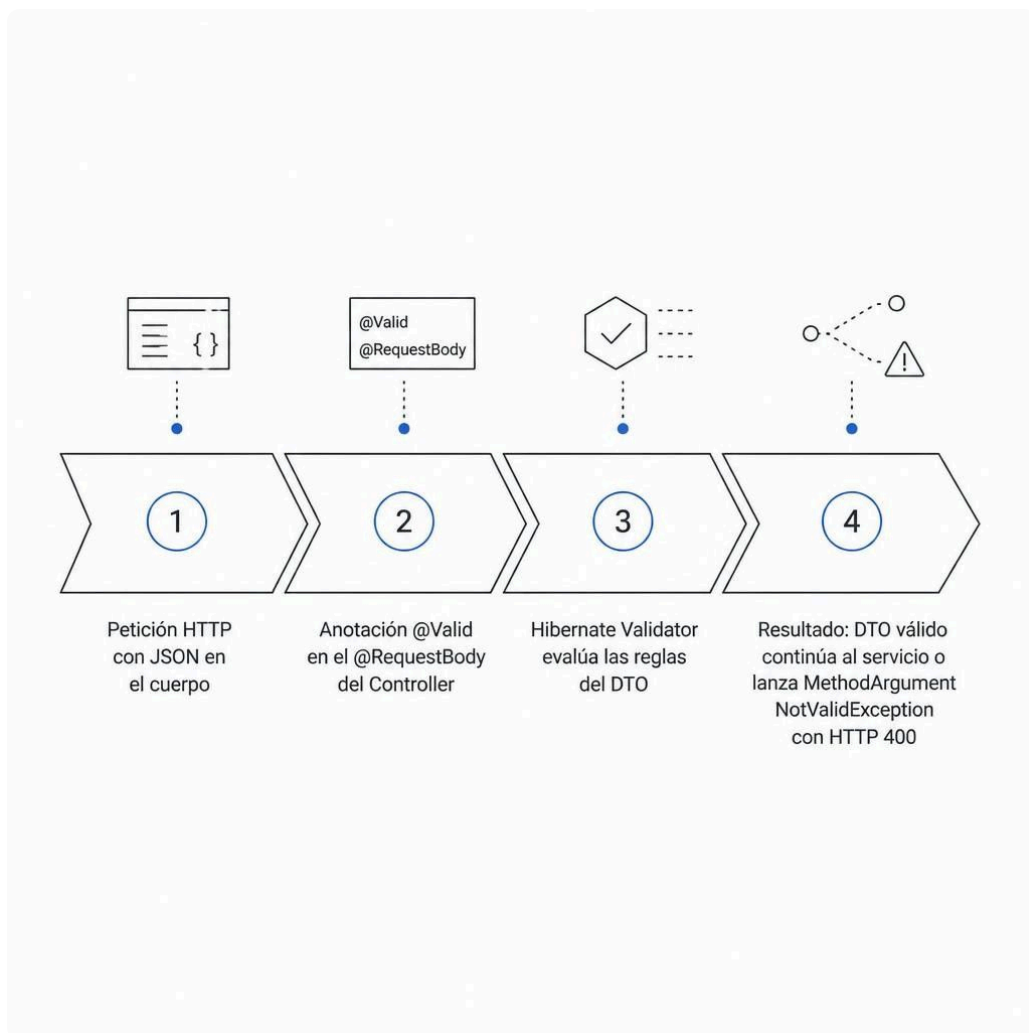
Validar los datos en los DTOs antes de que lleguen al servicio. Las reglas se declaran como anotaciones sobre los campos.



Pilar 2: @ControllerAdvice

Centralizar el manejo de todas las excepciones en una sola clase. Un único punto de control para toda la API.

¿Qué es Bean Validation? (JSR-380)



Conceptos clave

- Estándar de Java (JSR-380) para definir reglas de validación mediante anotaciones
- Spring Boot incluye **Hibernate Validator** por defecto — sin dependencias extra
- Al usar `@Valid` en un `@RequestBody`, Spring valida automáticamente
- Si algo falla, lanza `MethodArgumentNotValidException` con detalles del campo inválido

Anotaciones de Validación: Tu Caja de Herramientas

Anotación	Aplica a	Descripción	Ejemplo en Patitas Felices
@NotBlank	String	No nulo, no vacío, no solo espacios	nombre del dueño
@Email	String	Formato de email válido	email del dueño
@FutureOrPresent	Fecha	Hoy o en el futuro	fecha del turno
@Pattern	String	Expresión regular personalizada	matrícula MV-XXXX

Cada anotación acepta `message = "mensaje personalizado"` para respuestas de error claras.

Ejemplo Práctico: Validando un DTO

```
public class DuenoDTO {  
  
    @NotBlank(message = "El nombre es obligatorio")  
    private String nombre;  
  
    @Email(message = "Debe ser un email válido")  
    private String email;  
  
    @Pattern(regexp = "^\\+?[0-9]{8,15}$",  
            message = "Teléfono inválido")  
    private String telefono;  
  
    // getters y setters  
}
```

La validación vive en el DTO

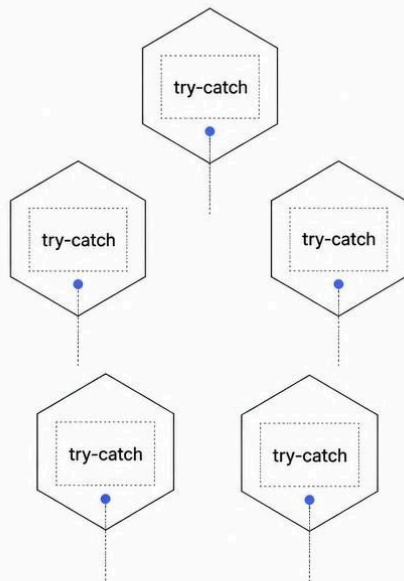
El DTO es el **contrato de entrada** de la API. Las reglas de validación declaradas aquí son la primera línea de defensa.

- ❏ Con `@Valid` en el Controller, Spring valida automáticamente antes de ejecutar cualquier lógica de negocio.

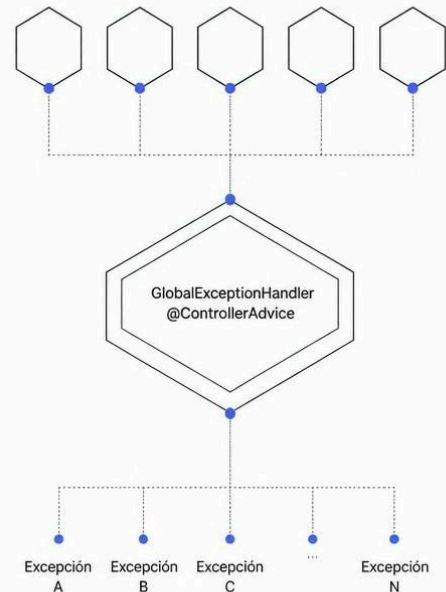
Del try-catch Disperso al Código Centralizado

Comparación de dos enfoques de manejo de errores:

Enfoque disperso (mala práctica): cada Controller tiene su propio try-catch, código repetitivo en 5+ clases, difícil de mantener y modificar



Enfoque centralizado (buena práctica): una sola clase GlobalExceptionHandler con @ControllerAdvice, captura todas las excepciones de la API, un único punto de modificación



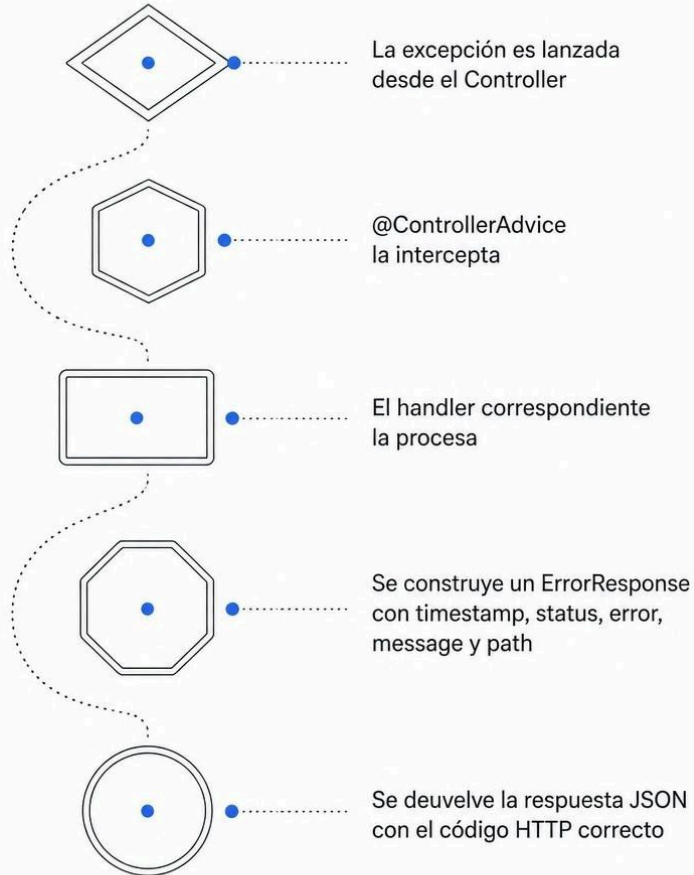
El problema del código disperso

Cada Controller tiene su propio try-catch para `ResourceNotFoundException`. Esto genera código repetitivo, inconsistente y difícil de mantener.

La solución: @ControllerAdvice

Una única clase centraliza toda la lógica de errores. Si cambia el formato de respuesta, se modifica en **un solo lugar** y afecta a toda la API.

Construyendo el GlobalExceptionHandler



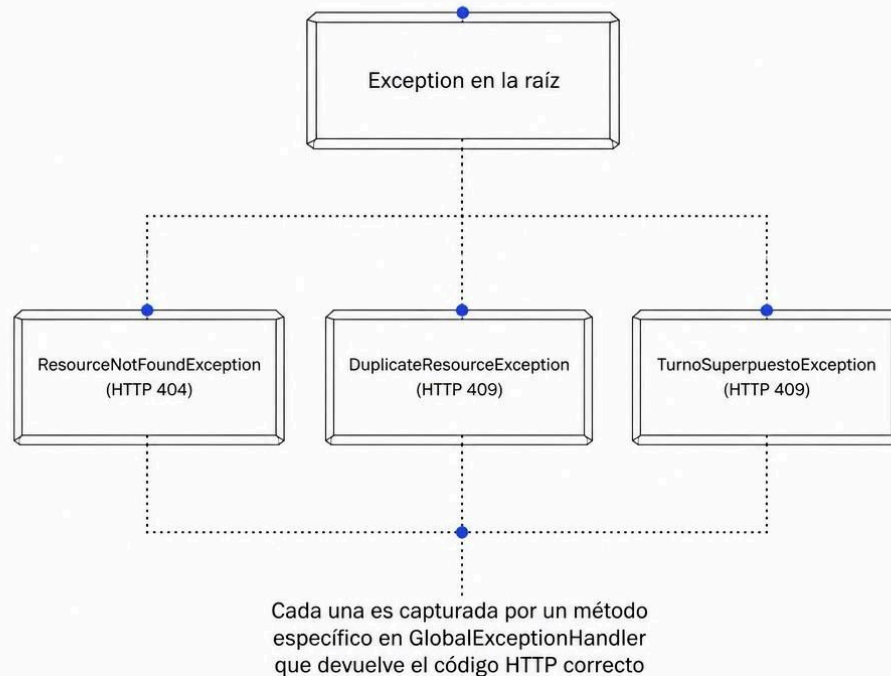
ErrorResponse — estructura estándar

Todas las respuestas de error comparten los campos: `timestamp`, `status`, `error`, `message` y `path`.

Métodos del handler

- `handleValidationExceptions()` → **HTTP 400**
- `handleResourceNotFound()` → **HTTP 404**
- `handleDuplicateResource()` → **HTTP 409**
- `handleGenericException()` → **HTTP 500** (solo errores inesperados)

Excepciones Tipadas vs. RuntimeException Genérica



¿Por qué crear excepciones propias?

Usar `throw new RuntimeException("mensaje")` no permite distinguir un error de negocio de un fallo del sistema.

Con excepciones tipadas como `DuplicateResourceException` o `TurnoSuperpuestoException`, el `GlobalExceptionHandler` puede devolver el código HTTP correcto: **409 Conflict** para duplicados, **400** para validaciones, **404** para no encontrados.

Resumen: ¿Qué Logramos?

-  Bean Validation en todos los DTOs de entrada
-  @Valid activado en todos los Controladores
-  GlobalExceptionHandler centralizando el manejo de errores
-  Excepciones tipadas para errores de negocio
-  Respuestas de error consistentes con ErrorResponse
-  Controllers más limpios, sin try-catch

🕒 **Reflexión final:** Ningún endpoint devuelve HTTP 500 por datos inválidos del cliente. **Próximo Sprint:** Testing con JUnit, Mockito y MockMvc.